

Universidad ORT Uruguay

Facultad de Ingeniería

Bernard Wand Polak

Diseño de Aplicaciones 2

Evidencia de TDD

Joaquin Rodríguez (231355)

Agustín Madrid (270635)

Juan Diego Meneses (224304)

2024

Índice

Clean Code	2
Descripción de estrategia	3
Informe de cobertura de pruebas	3
Separación de los proyectos de Test del resto	4
Funcionalidades especificadas (*)	5
Evidencia de TDD:	5

Clean Code

Durante todo el proyecto se siguieron las prácticas recomendadas por el Clean Code. El código se puede leer y entender con facilidad, por esta razón es que no incluimos comentarios explicativos como parte del código.

Además, en la solución se utilizan nombres nemotécnicos, tanto para variables o para métodos, para que no haya confusiones ni ambigüedades. Todas las funciones tienen nombres autoexplicativos para su rápida identificación y entendimiento.

Tomemos de ejemplo la entidad Company. Fácilmente podemos ver que una Compañía está formada por un id, nombre, rut, logo y dueño.

```
public class Company
{
    15 references
    public Guid Id { get; set; }
    20 references
    public string Name { get; set; } = string.Empty;
    17 references
    public string RUT { get; set; } = string.Empty;
    10 references
    public string Logo { get; set; } = string.Empty;
    24 references
    public required User Owner { get; set; }
}
```

También se intentó reutilizar el código al máximo, para no tener clases muy largas y métodos engorrosos. Al mismo tiempo, logramos que nuestros métodos hagan solo 1 cosa y que nos queden de pocas líneas.

Por ejemplo en nuestra función DeleteUser, en vez de hacer un chequeo manual si existe el usuario a borrar y volver a escribir el código, llamamos a otra función ya existente, ExistUser, para que lo haga.

```
5 references
public User DeleteUser(Guid userId)
{
    if (!_userRepository.ExistUser(userId))
    {
        throw new NotValidDataException("User does not exist");
    }
    return _userRepository.DeleteUser(userId);
}
```

Descripción de estrategia

Después de días de discusiones sobre los pros y contras de seguir la estrategia Inside-Out o Outside-In, se optó por utilizar Outside-In. Esto significa que comenzamos desarrollando las funcionalidades desde la capa más externa, la **WebAPI**, y luego avanzamos hacia las capas internas, como la lógica de negocios (**BusinessLogic**) y, finalmente, el acceso a datos (**DataAccess**).

La razón principal para elegir Outside-In fue que esta estrategia nos permitió enfocarnos primero en cómo el sistema debería comportarse desde la perspectiva del usuario final, garantizando que las interacciones con la API cumplieran con los requerimientos antes de adentrarnos en los detalles de implementación. Además, este enfoque nos ayudó a definir las interfaces entre capas desde el inicio, asegurando que cada módulo interno se desarrollará en función de las necesidades claramente identificadas por la API.

Al seguir Outside-In, desarrollamos pruebas unitarias que simulan las llamadas a los controladores de la WebAPI. Estas pruebas nos permitieron validar los casos de uso más importantes desde el principio y verificar que la API devolviera respuestas válidas antes de implementar los detalles en las capas internas.

Informe de cobertura de pruebas

Name	Covered	Uncovered	Coverable	Total		Line coverage	Covered	Total		Branch coverage
+ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
+ DataAccess	404	48	452	702	89.3%		143	234	61.1%	
+ Domain	69	1	70	142	98.5%		0	0		
+ Models	158	14	172	350	91.8%		1	2	50%	
+ WebApi	408	55	463	836	88.1%		61	80	76.2%	

Assemblies:	9
Classes:	67
Files:	64
Covered lines:	1305
Uncovered lines:	142
Coverable lines:	1447
Total lines:	2533
Line coverage:	90.1% (1305 of 1447)
Covered branches:	266
Total branches:	386
Branch coverage:	68.9% (266 of 386)

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total		▼ Line coverage	▼ Covered	▼ Total		▼ Branch coverage
─ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
BusinessLogic.CompanyLogic	31	2	33	59	93.9%		13	14	92.8%	
BusinessLogic.ConflictException	3	0	3	8	100%		0	0		
BusinessLogic.DeviceLogic	46	6	52	84	88.4%		15	18	83.3%	
BusinessLogic.HomeLogic	57	3	60	102	95%		6	8	75%	
BusinessLogic.MemberSettingLogic	28	0	28	55	100%		0	0		
BusinessLogic.NotValidDataException	3	0	3	8	100%		0	0		
BusinessLogic.SessionService	36	0	36	66	100%		8	8	100%	
BusinessLogic.UserLogic	62	9	71	117	87.3%		19	22	86.3%	

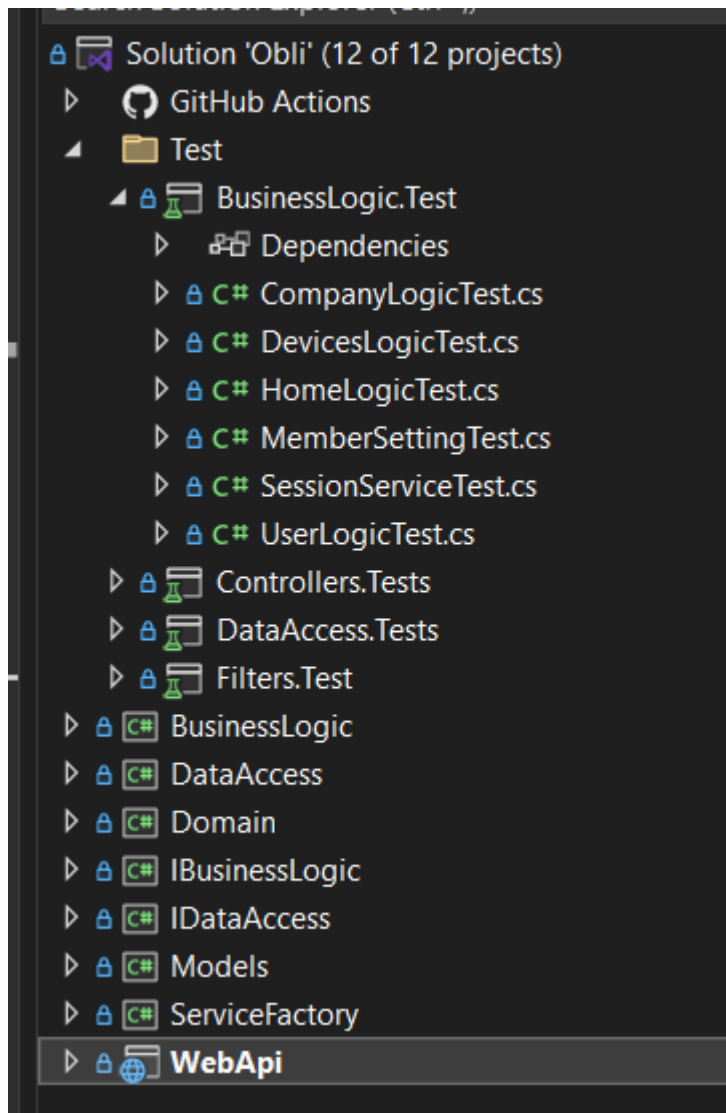
▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total		▼ Line coverage	▼ Covered	▼ Total		▼ Branch coverage
+ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
─ DataAccess	404	48	452	702	89.3%		143	234	61.1%	
DataAccess.NotificationRepository	80	2	82	114	97.5%		31	44	70.4%	
DataAccess.Repositories.CompanyRepository	22	5	27	50	81.4%		8	16	50%	
DataAccess.Repositories.DeviceRepository	38	0	38	69	100%		11	18	61.1%	
DataAccess.Repositories.EntityNotFoundException	3	0	3	8	100%		0	0		
DataAccess.Repositories.HomeRepository	97	0	97	141	100%		34	52	65.3%	
DataAccess.Repositories.MemberSettingRepository	68	23	91	146	74.7%		30	56	53.5%	
DataAccess.Repositories.UserRepository	96	18	114	174	84.2%		29	48	60.4%	

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total		▼ Line coverage	▼ Covered	▼ Total		▼ Branch coverage
+ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
+ DataAccess	404	48	452	702	89.3%		143	234	61.1%	
─ Domain	69	1	70	142	98.5%		0	0		
Domain.Camera	4	0	4	9	100%		0	0		
Domain.Company	6	0	6	14	100%		0	0		
Domain.Device	8	0	8	16	100%		0	0		
Domain.Home	10	0	10	16	100%		0	0		
Domain.HomeDevice	5	0	5	10	100%		0	0		
Domain.MemberSetting	4	0	4	9	100%		0	0		
Domain.Notification	8	0	8	13	100%		0	0		
Domain.Permission	3	0	3	12	100%		0	0		
Domain.PermissionKey	3	1	4	8	75%		0	0		
Domain.Role	3	0	3	9	100%		0	0		
Domain.Session	4	0	4	10	100%		0	0		
Domain.User	11	0	11	16	100%		0	0		

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total		▼ Line coverage	▼ Covered	▼ Total		▼ Branch coverage
+ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
+ DataAccess	404	48	452	702	89.3%		143	234	61.1%	
+ Domain	69	1	70	142	98.5%		0	0		
– Models	158	14	172	350	91.8%		1	2	50%	
Models.AddMemberRequest	1	0	1	6	100%		0	0		
Models.AddMemberResponse	2	0	2	10	100%		0	0		
Models.AdminRequest	7	0	7	16	100%		0	0		
Models.AdminResponse	3	0	3	8	100%		0	0		
Models.AuthenticationResult	0	2	2	7	0%		0	0		
Models.CameraRequest	20	0	20	29	100%		0	0		
Models.CameraResponse	11	0	11	19	100%		0	0		
Models.CompanyOwnerRequest	7	0	7	16	100%		0	0		
Models.CompanyOwnerResponse	3	0	3	8	100%		0	0		
Models.CompanyRequest	19	0	19	29	100%		1	2	50%	
Models.CompanyResponse	13	0	13	22	100%		0	0		
Models.DeviceRequest	5	12	17	26	29.4%		0	0		
Models.DeviceResponse	15	0	15	24	100%		0	0		
Models.GetHomeMembersResponse	2	0	2	8	100%		0	0		
Models.GetUserResponse	5	0	5	12	100%		0	0		
Models.HomeDeviceRequest	1	0	1	6	100%		0	0		
Models.HomeDeviceResponse	2	0	2	9	100%		0	0		
Models.HomeOwnerRequest	7	0	7	17	100%		0	0		
Models.HomeOwnerResponse	3	0	3	8	100%		0	0		
Models.HomeRequest	14	0	14	23	100%		0	0		
Models.HomeResponse	6	0	6	14	100%		0	0		
Models.LoginRequest	2	0	2	7	100%		0	0		
Models.LoginResponse	7	0	7	13	100%		0	0		
Models.PermissionRequest	2	0	2	7	100%		0	0		
Models.SensorRequest	1	0	1	6	100%		0	0		

▼ Name	▼ Covered	▼ Uncovered	▼ Coverable	▼ Total		▼ Line coverage	▼ Covered	▼ Total		▼ Branch coverage
+ BusinessLogic	266	20	286	499	93%		61	70	87.1%	
+ DataAccess	404	48	452	702	89.3%		143	234	61.1%	
+ Domain	69	1	70	142	98.5%		0	0		
+ Models	158	14	172	350	91.8%		1	2	50%	
– WebApi	408	55	463	836	88.1%		61	80	76.2%	
WebApi.Controllers.AdminController	19	0	19	43	100%		0	0		
WebApi.Controllers.CompanyController	12	0	12	32	100%		0	0		
WebApi.Controllers.CompanyOwnerController	8	0	8	28	100%		0	0		
WebApi.Controllers.DevicesController	19	7	26	60	73%		0	0		
WebApi.Controllers.HomeController	91	7	98	179	92.8%		2	6	33.3%	
WebApi.Controllers.HomeOwnerController	8	0	8	25	100%		0	0		
WebApi.Controllers.LoginController	16	2	18	40	88.8%		3	6	50%	
WebApi.Controllers.UserController	60	0	60	112	100%		0	0		
WebApi.Filters.AuthenticationFilterAttribute	45	22	67	104	67.1%		5	6	83.3%	
WebApi.Filters.AuthorizationFilterAttribute	100	17	117	168	85.4%		49	60	81.6%	
WebApi.Filters.ExceptionFilter	30	0	30	45	100%		2	2	100%	

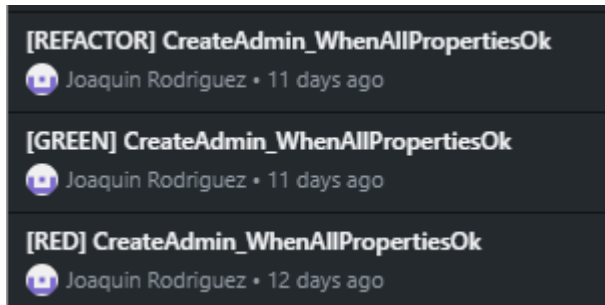
Separación de los proyectos de Test del resto



Funcionalidades especificadas (*)

Evidencia de TDD y cobertura de pruebas:

Mantenimiento de cuentas de administrador ()*



This screenshot shows the output of a test runner, displaying a list of tests and their execution times. All tests are marked with a green checkmark, indicating they passed. The tests are organized into a hierarchical structure with expandable/collapsible icons.

Test Name	Execution Time
BusinessLogic.Test (16)	703 ms
BusinessLogic.Test (16)	703 ms
UserLogicTest (16)	703 ms
AddCompanyToUserTest	2 ms
CreateAdminTest	1 ms
CreateCompanyOwner_WhenU...	< 1 ms
CreateCompanyOwnerTest	659 ms
CreateHomeOwner_WhenUser...	1 ms
CreateHomeOwnerTest	1 ms
CreateUser_WhenUserHasNoE...	1 ms
CreateUser_WhenUserHasNoN...	< 1 ms
DeleteNotExistUserTest_WhenU...	21 ms
DeleteUserTest	4 ms
DeleteUserTest_WhenUserExist	2 ms
ExistUserTest	2 ms
ExistUserTest_WhenUserExist	< 1 ms

Creación de una empresa (*)

[REFACTOR] Interfaces in right place Agustin Madrid • 10 days ago
[REFACTOR] Models in right place Agustin Madrid • 10 days ago
[GREEN] Add Company to CompanyOwner Agustin Madrid • 10 days ago
[RED] Add Company to CompanyOwner Agustin Madrid • 10 days ago

✓ BusinessLogic.Test (20)	820 ms
✓ BusinessLogic.Test (20)	820 ms
✓ CompanyLogicTest (4)	117 ms
✓ CreateCompany_WhenPropertiesAreWrong	14 ms
✓ CreateCompany_WhenTheOwnerHasAlreadyACompany	2 ms
✓ CreateCompanyTest_WhenAllPropertiesOk	97 ms
✓ GetCompaniesTest_WhenAllPropertiesOk	4 ms

Detección de movimiento (*)

[REFACTOR] Now we return HomeDeviceResponse at AddingDevices Agus-Madrid • 2 days ago
[GREEN] NotificationRepository CreateNotiCameraShouldAddToDv Agus-Madrid • 2 days ago
[GREEN] NotificationRepository CreateNotiCameraShouldntCreate_NoDe... Agus-Madrid • 2 days ago
[GREEN] NotificationRepository CreateNotiCameraShouldCreateOnlyFor... Agus-Madrid • 2 days ago
[GREEN] CreateNotificationCamera_WhenAllPropertiesOk Agus-Madrid • 2 days ago
[RED] CreateNotificationCamera_WhenAllPropertiesOk Agus-Madrid • 2 days ago

Asociar dispositivos al hogar (*)

[REFACTOR] Seed data, little modifications

 Agustin Madrid • 2 days ago

[GREEN] GetHomeDevices_WhenHomeHasDevices

 Agustin Madrid • 2 days ago

[RED] GetHomeDevices_WhenHomeHasDevices

 Agustin Madrid • 2 days ago

[GREEN] AddDevicToHome_WhenAllPropertiesOK

 Agustin Madrid • 2 days ago